

BINV2150-A – Javascript : avancé

Séance 11 – Événements & état

Jérome Plumat & Sébastien Strebelle

Haut École Léonard de Vinci

2025-2026



Haute École Léonard de Vinci

Gestionnaire d'événements



- Propriété des éléments HTML
- Nom : on + nom de l'évènement (onClick, onMouseOver, onSubmit)
- Valeur : fonction exécutée lors de l'évènement
- Un argument optionnel : l'évènement
- Une ligne de code : fonction anonyme
- Plusieurs lignes : fonction nommée (handle + nom de l'évènement)

```
1 <button onClick={() ⇒ console.log("Clicked")}>Click me</button>
2
3
4 const handleClick = (event: MouseEvent) ⇒ {
5     console.log(event);
6     // réagir a l'évènement
7 }
8
9 <button onClick={handleClick}>Click me</button>
```

Problème de l'état



- Comment réagir aux évènements ?
- Modifier des variables
- Sauf que... l'interface ne se met pas à jour
- Il faut utiliser le hook useState
- useState retourne un tableau de deux éléments :
  - La variable d'état
  - La fonction pour modifier la variable d'état
- La fonction de modification prend en argument la nouvelle valeur ou une fonction qui retourne la nouvelle valeur

Problème de l'état



Live demo

Variable d'état



- Variable crée avec `useState`
- useState retourne un tableau de deux éléments :
  - La variable d'état
  - La fonction pour modifier la variable d'état
- La variable d'état est modifiée par la fonction de modification
- La modification de la variable d'état provoque un nouveau rendu
- La fonction de modification prend en argument la nouvelle valeur ou une fonction qui retourne la nouvelle valeur

```
1 const [count, setCount] = useState(0);
2
3 <button onClick={() ⇒ setCount(count + 1)}>{count} clicks</button>
```

hook useState



- `useState` est un hook
- hook = fonction qui permet d'utiliser des fonctionnalités de React
- Lors du premier rendu, la valeur initiale est utilisée
- À chaque modification de l'état, le composant est re-rendu

Re-rendu



- React exécute à nouveau la fonction du composant
- Le JSX retourné est comparé au JSX précédent
- Seules les différences sont appliquées au DOM

Programmation déclarative



- Autre paradigme de programmation (<> impérative)
- On déclare ce que l'on veut afficher en fonction de l'état
- React se charge de mettre à jour l'interface lorsque l'état change
- Avantages :
  - Code plus simple et plus lisible
  - Meilleure séparation des préoccupations

Résumé



- Les gestionnaires d'évènements permettent de réagir aux actions
- Le hook useState permet de gérer l'état dans les composants
- Lorsque l'état change, le composant est re-rendu
- La programmation déclarative facilite la gestion de l'interface